



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Parametric Polymorphism in C++, Java, and C#

Generics vs Templates and Metaprogramming Power

Course:

Advanced Programming

Student:

Elia Leonardi

ACADEMIC YEAR 2025/2026

Contents

- 1 Introduction 2**
 - 1.1 Prompt Output Comparing 2
 - 1.2 Prompt Output Resources 2
 - 1.3 Output Script 2
 - 1.4 AI verification 2
- 2 Task 1 3**
 - 0 Introduction 3
 - 0.5 Performance 3
 - 1 C++ Templates 3
 - 1.1 Instantiation Time 3

Chapter 1

Introduction

For the writing of the report, is use a AI local model with LM studio with qwen2.5-coder-7b-instruct, this is helpfull for the intellsense of latex and layout of the report, but it doesn't interfere with the content of the report. The verification of the report procede in 3 steps.

1.1 Prompt Output Comparing

Every prompt on the same language produce an output that contains a fraction of the information to other prompts of the same language, this repetition helpt to make a first check of eventualy error missing or wrong information, eventualy allucinations and in generale the quality of the output.

1.2 Prompt Output Resources

Each prompt output is follow by a list of resources that are used to generate the output, as subset of the resources is used to verify the correctness of the output.

1.3 Output Script

Generated scripts are verified by running them in a controlled environment, checking for errors, and ensuring they produce the expected results. No AI model is involved in this step, as it is a manual verification process.

1.4 AI verification

An AI model is used on the resoult of the previous steps of the verification, if there is any error or missing information, the process of verification is repeated until the output is correct and complete. The AI model used for this step Claude Sonnet 5.

Chapter 2

Task 1

0 Introduction

0.5 Performance

Imprecision:

Homogeneous implementations generally produce smaller binaries and
→ reduce compilation
time because generic definitions are compiled only once.

This statement is presented as a general rule, but it is too absolute: the reduction in compilation time holds for the type-checking and code-generation phase specific to the generic definitions themselves, but it does not account for additional compilation work introduced by erasure, such as the insertion of runtime casts, boxing/unboxing operations, or bridge methods [1]. The claim is therefore corrected as follows:

Homogeneous implementations generally produce smaller binaries
→ because generic definitions
are compiled only once; compilation time is often reduced as well,
→ although this benefit can
be partially offset by the additional work required to insert runtime
→ casts, boxing/unboxing
operations, or bridge methods.

1 C++ Templates

1.1 Instantiation Time

1.1.2 Instantiation Trigger

Imprecisions:

In every case, code generation begins only once the complete set of
→ arguments
is known.

This statement is presented as universally true for every instantiation trigger mentioned (implicit instantiation, explicit instantiation, explicit specialization), but it does not hold for *explicit instantiation declarations* (`extern template`), where the template arguments are fully known yet code generation in that translation unit is deliberately suppressed and deferred to another translation unit containing the corresponding explicit instantiation definition [3]. The claim is therefore corrected as follows:

In most cases, code generation begins once the complete set of
→ arguments is
known; the exception is the explicit instantiation declaration
→ (`extern
template`), which suppresses code generation in the current
→ translation unit
even though the arguments are fully known, deferring it to a
→ translation unit
containing the corresponding explicit instantiation definition.

1.1.6 Completion of the Instantiation Process

Imprecisions:

Instantiation terminates entirely before linking: by the time object
→ files are
generated, every specialization required by the translation unit
→ already exists
as compiled code, and the final linked executable contains all of
→ them.

This statement correctly describes what happens within a single translation unit, but it obscures the role of the linker when the same instantiation is required by multiple translation units: each translation unit independently generates its own copy of the specialization, and it is the linker, not the compiler, that discards the duplicate copies and retains a single definition in the final executable, typically through weak symbols or COMDAT sections [2]. The claim is therefore corrected as follows:

Instantiation is triggered and resolved entirely during compilation:
→ by the
time object files are generated, every specialization required by a
→ given
translation unit already exists as compiled code. When the same
→ instantiation
is required by multiple translation units, each one generates its own
→ copy,
and it is the linker that discards the duplicates and retains a
→ single
definition in the final executable.

Bibliography

- [1] Gilad Bracha. Generics in the java programming language, 2004.
- [2] Itanium C++ ABI Project. Itanium c++ abi: C++ abi specification, 2024. C++ Application Binary Interface specification including name mangling and RTTI representation.
- [3] David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.